

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

Practical No : 7,8

Aim :

7. Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.
8. Cluster a dataset into two groups using k-means clustering in Breeze.

CODE 7 :

```
import breeze.linalg.{DenseVector, euclideanDistance}
import scala.io.Source

object omwala_M2_pract7 {

  case class DataPoint(
    features: DenseVector[Double],
    label: String,
    playerId: String
  )

  def main(args: Array[String]): Unit = {

    println("Om Wala S119")
    println("K-Nearest Neighbor Classification")
    println("-----")

    val filePath = "src/main/resources/Player_Attributes.csv"

    val source = Source.fromFile(filePath)
    val lines = source.getLines().toList
    source.close()

    val headers = lines.head.split(",").map(_.trim)

    val featureColumns = Array(
      "potential",
      "crossing",
      "finishing",
      "dribbling",
      "short_passing"
    )

    val featureIndexes =
      featureColumns.map(column => headers.indexOf(column))
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val overallIndex =
  headers.indexOf("overall_rating")

val idIndex =
  headers.indexOf("id")

if (featureIndexes.contains(-1)) {
  println("Error: Feature column not found.")
  return
}

if (overallIndex == -1) {
  println("Error: overall_rating column not found.")
  return
}

if (idIndex == -1) {
  println("Error: id column not found.")
  return
}

val dataset = lines.tail.flatMap { line =>

  val values = line.split(",", -1).map(_.trim)

  try {

    val features = DenseVector(
      featureIndexes.map(index => values(index).toDouble): _*
    )

    val overallRating =
      values(overallIndex).toDouble

    val playerId =
      values(idIndex)

    val label =
      if (overallRating < 60) {
        "Low"
      } else if (overallRating < 75) {
        "Medium"
      } else {
        "High"
      }
  }
}
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
Some(
  DataPoint(
    features,
    label,
    playerId
  )
)

} catch {
  case _: Exception => None
}
}

println(
  s"\nTotal records loaded: ${dataset.length}"
)

println("\nFeatures used:")
featureColumns.foreach(column =>
  println(s"- $column")
)

println("\nClass:")
println("Low   : Overall Rating < 60")
println("Medium : Overall Rating 60-74")
println("High   : Overall Rating >= 75")

print("\nEnter K value: ")
val k = scala.io.StdIn.readInt()

if (k <= 0 || k >= dataset.length) {
  println("Invalid K value.")
  return
}

print("Enter Player ID to classify: ")
val inputId = scala.io.StdIn.readLine()

// Find selected player
val selectedPlayer =
  dataset.find(_.playerId == inputId)

if (selectedPlayer.isEmpty) {
  println("Player ID not found.")
  return
}
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val player = selectedPlayer.get

println("\nSelected Player")
println("-----")
println(s"Player ID: ${player.playerId}")
println(s"Features: ${player.features}")
println(s"Actual Class: ${player.label}")

val distances =
  dataset
    .filter(_ playerId != inputId)
    .map { point =>

      val distance =
        euclideanDistance(
          player.features,
          point.features
        )

      (distance, point.label)
    }

val nearestNeighbors =
  distances
    .sortBy(_._1)
    .take(k)

println("\nNearest Neighbors")
println("-----")

nearestNeighbors.zipWithIndex.foreach {
  case ((distance, label), index) =>

    println(
      s"${index + 1}. " +
      f"Distance = $distance%.4f" +
      s", Class = $label"
    )
}

val votes =
  nearestNeighbors
    .groupBy(_._2)
    .map {
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
case (label, records) =>
  (label, records.size)
}

val predictedLabel =
  votes.maxBy(_._2)._1

println("\nVoting Result")
println("-----")

votes.foreach {
  case (label, count) =>
    println(
      s"$label = $count vote(s)"
    )
}

println("\nClassification Result")
println("-----")

println(s"K Value: $k")
println(s"Actual Class: ${player.label}")
println(s"Predicted Class: $predictedLabel")

if (player.label == predictedLabel) {
  println("Prediction: Correct")
} else {
  println("Prediction: Incorrect")
}
}
}
```

OUTPUT :

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
Run  omwala_M2_pract7 x

"\"C:\\Program Files\\Java\\jdk-21.0.11\\bin\\java.exe\" ...
Om Wala S119
K-Nearest Neighbor Classification
-----

Total records loaded: 183142

Features used:
- potential
- crossing
- finishing
- dribbling
- short_passing

Class:
Low    : Overall Rating < 60
Medium : Overall Rating 60-74
High   : Overall Rating >= 75

Enter K value: 10
Enter Player ID to classify: 119

Selected Player
-----
Player ID: 119
Features: DenseVector(78.0, 77.0, 72.0, 78.0, 80.0)
Actual Class: High

Nearest Neighbors
-----
1. Distance = 0.0000, Class = High
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
2. Distance = 0.0000, Class = High
3. Distance = 1.0000, Class = High
4. Distance = 1.4142, Class = High
5. Distance = 1.4142, Class = High
6. Distance = 1.4142, Class = High
7. Distance = 1.4142, Class = High
8. Distance = 1.4142, Class = High
9. Distance = 1.7321, Class = High
10. Distance = 1.7321, Class = High
```

Voting Result

High = 10 vote(s)

Classification Result

K Value: 10

Actual Class: High

Predicted Class: High

Prediction: Correct

Process finished with exit code 0

CODE 8 :

```
import breeze.linalg._
import scala.io.Source
import scala.util.Random
```

```
object omwala_M2_pract8 {
```

```
  def main(args: Array[String]): Unit = {
```

```
    println("Om Wala S119")
    println("K-Means Clustering using Player Attributes")
    println("-----")
```

```
    val filePath = "src/main/resources/Player_Attributes.csv"
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val source = Source.fromFile(filePath)
val lines = source.getLines().toList
source.close()

val headers = lines.head.split(",").map(_.trim)

val featureColumns = Array(
  "overall_rating",
  "potential",
  "crossing",
  "finishing",
  "dribbling"
)

val featureIndexes =
  featureColumns.map(column => headers.indexOf(column))

if (featureIndexes.contains(-1)) {
  println("Required columns were not found.")
  return
}

val dataRows = lines.tail.flatMap { line =>

  val values = line.split(",", -1).map(_.trim)

  try {
    Some(
      featureIndexes.map(index => values(index).toDouble)
    )
  } catch {
    case _: Exception => None
  }
}

val maxSamples = 5000

val selectedRows =
  Random.shuffle(dataRows).take(maxSamples)

val numSamples = selectedRows.length
val numFeatures = featureColumns.length

val data =
  DenseMatrix.zeros[Double](numSamples, numFeatures)

for (i <- 0 until numSamples) {
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
for (j <- 0 until numFeatures) {
  data(i, j) = selectedRows(i)(j)
}
}

println(
  s"Dataset loaded with $numSamples samples and $numFeatures features."
)

println("\nFeatures used:")

featureColumns.foreach { column =>
  println(column)
}

print("\nEnter number of clusters (K): ")
val k = scala.io.StdIn.readInt()

if (k <= 0 || k > numSamples) {
  println("Invalid value of K.")
  return
}

val maxIterations = 100

var centroids =
  DenseMatrix.zeros[Double](k, numFeatures)

val randomIndices =
  Random.shuffle((0 until numSamples).toList).take(k)

for (i <- 0 until k) {
  for (j <- 0 until numFeatures) {
    centroids(i, j) =
      data(randomIndices(i), j)
  }
}

println("\nInitial Centroids:")
println(centroids)

var assignments =
  DenseVector.zeros[Int](numSamples)

var previousAssignments =
  DenseVector.fill(numSamples)(-1)

var iteration = 0
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
var converged = false

while (iteration < maxIterations && !converged) {

  for (i <- 0 until numSamples) {

    val point =
      DenseVector(
        (0 until numFeatures)
          .map(j => data(i, j))
          .toArray
      )

    var minDistance =
      Double.MaxValue

    var closestCentroid =
      -1

    for (j <- 0 until k) {

      val centroid =
        DenseVector(
          (0 until numFeatures)
            .map(column => centroids(j, column))
            .toArray
        )

      val distance =
        euclideanDistance(point, centroid)

      if (distance < minDistance) {
        minDistance = distance
        closestCentroid = j
      }
    }

    assignments(i) =
      closestCentroid
  }

  converged =
    assignments == previousAssignments

  previousAssignments =
    assignments.copy

  val newCentroids =
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
DenseMatrix.zeros[Double](k, numFeatures)

val clusterCounts =
  DenseVector.zeros[Int](k)

for (i <- 0 until numSamples) {

  val clusterId =
    assignments(i)

  for (j <- 0 until numFeatures) {

    newCentroids(clusterId, j) +=
      data(i, j)
  }

  clusterCounts(clusterId) += 1
}

for (i <- 0 until k) {

  if (clusterCounts(i) > 0) {

    for (j <- 0 until numFeatures) {

      newCentroids(i, j) =
        newCentroids(i, j) /
        clusterCounts(i).toDouble
    }
  }
}

centroids =
  newCentroids

iteration += 1

println(
  s"Iteration $iteration completed."
)

println("\n--- Final Results ---")

println(
  s"K-Means converged in $iteration iterations."
)
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
println("\nFinal Centroids:")

for (i <- 0 until k) {

  print(s"Cluster ${i + 1}: ")

  for (j <- 0 until numFeatures) {

    print(
      f"${centroids(i, j)}%.2f "
    )
  }

  println()
}

println("\nCluster Sizes:")

for (i <- 0 until k) {

  val count =
    (0 until numSamples)
      .count(index => assignments(index) == i)

  println(
    s"Cluster ${i + 1}: $count players"
  )
}

println("\nFirst 20 Cluster Assignments:")

for (i <- 0 until math.min(20, numSamples)) {

  println(
    s"Player ${i + 1} -> Cluster ${assignments(i) + 1}"
  )
}
}
```

OUTPUT :

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
Run omwala_M2_pract8 x
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Om Wala $119
K-Means Clustering using Player Attributes
-----
Dataset Loaded with 5000 samples and 5 features.

Features used:
overall_rating
potential
crossing
finishing
dribbling

Enter number of clusters (K): 5

Initial Centroids:
63.0 65.0 25.0 25.0 25.0
67.0 78.0 14.0 13.0 13.0
72.0 74.0 50.0 55.0 48.0
67.0 75.0 42.0 68.0 68.0
87.0 91.0 79.0 91.0 85.0
Iteration 1 completed.
Iteration 2 completed.
Iteration 3 completed.
Iteration 4 completed.
Iteration 5 completed.
Iteration 6 completed.
Iteration 7 completed.
Iteration 8 completed.
Iteration 9 completed.
Iteration 10 completed.
```

```
Run omwala_M2_pract8 x
Iteration 10 completed.
Iteration 11 completed.
Iteration 12 completed.
Iteration 13 completed.
Iteration 14 completed.
Iteration 15 completed.
Iteration 16 completed.
Iteration 17 completed.
Iteration 18 completed.
Iteration 19 completed.
Iteration 20 completed.
Iteration 21 completed.
Iteration 22 completed.
Iteration 23 completed.
Iteration 24 completed.

--- Final Results ---
K-Means converged in 24 iterations.

Final Centroids:
Cluster 1: 64.50 70.42 43.64 33.08 45.45
Cluster 2: 67.05 71.86 20.56 18.80 20.72
Cluster 3: 68.12 72.90 64.07 41.42 62.23
Cluster 4: 65.40 71.15 51.38 63.01 63.55
Cluster 5: 74.72 78.42 71.12 67.89 75.87

Cluster Sizes:
Cluster 1: 828 players
Cluster 2: 523 players
Cluster 3: 1062 players
Cluster 4: 1211 players
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
Run  omwala_M2_pract8 x
Cluster Sizes:
Cluster 1: 828 players
Cluster 2: 523 players
Cluster 3: 1862 players
Cluster 4: 1211 players
Cluster 5: 1376 players

First 20 Cluster Assignments:
Player 1 -> Cluster 1
Player 2 -> Cluster 4
Player 3 -> Cluster 5
Player 4 -> Cluster 2
Player 5 -> Cluster 5
Player 6 -> Cluster 5
Player 7 -> Cluster 3
Player 8 -> Cluster 4
Player 9 -> Cluster 2
Player 10 -> Cluster 2
Player 11 -> Cluster 5
Player 12 -> Cluster 4
Player 13 -> Cluster 3
Player 14 -> Cluster 2
Player 15 -> Cluster 1
Player 16 -> Cluster 4
Player 17 -> Cluster 5
Player 18 -> Cluster 5
Player 19 -> Cluster 4
Player 20 -> Cluster 4

Process finished with exit code 0
```

Name : Om Wala
Roll No : S119